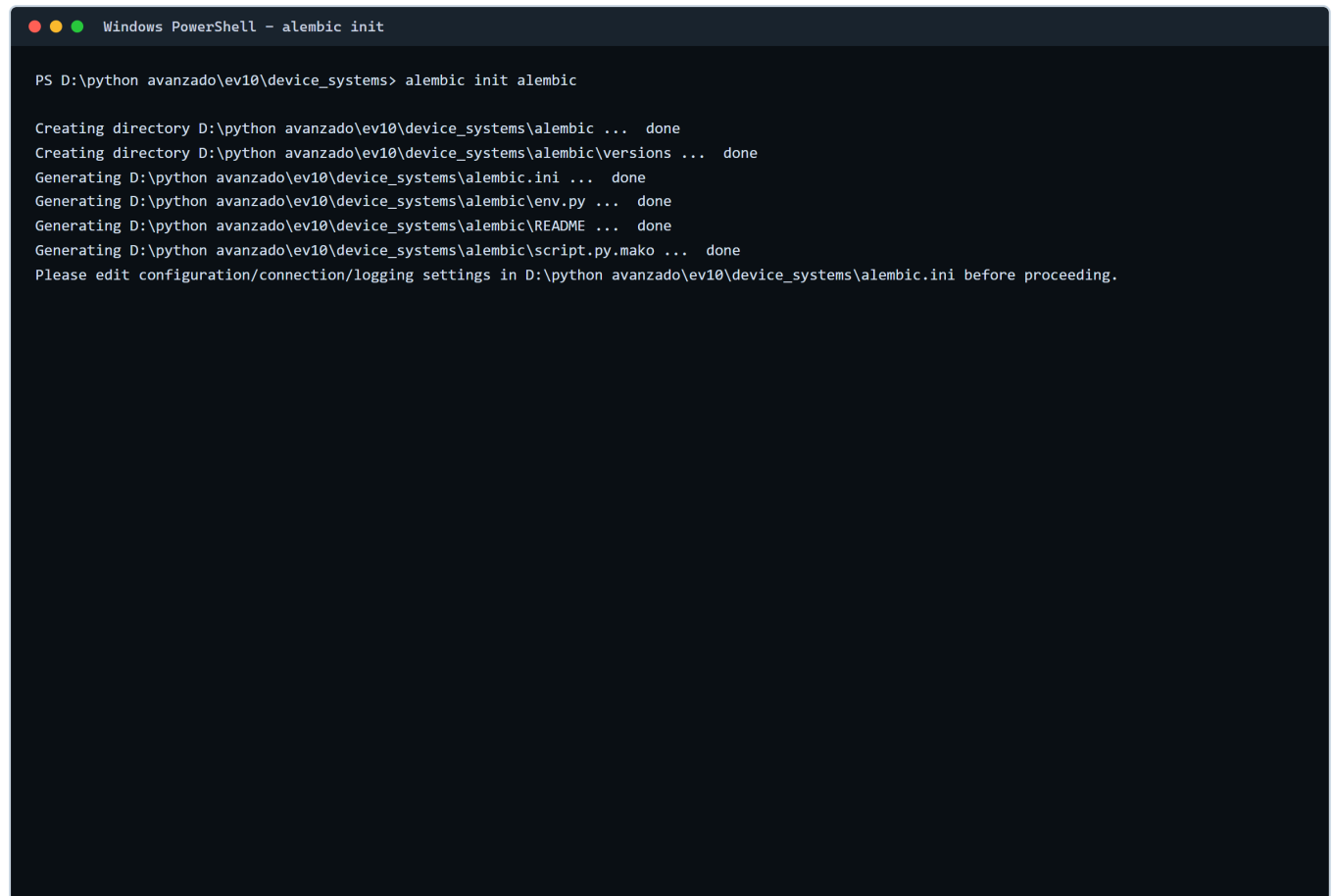


Evidencias EV10 — FastAPI Avanzado (Alembic, relaciones y joins)

Guia: [Proyecto-Final-v1] GA1-220501096-01-AA1-EV10 **Aprendiz:** Daniel Roman **Proyecto:** device_systems (v4.0.0) — código en ../device_systems/ **Rama:** device_systems_alembic_relaciones, unificada con main **Repositorio:** <https://github.com/daniel-roman345/python-avanzado->

1. Migraciones con Alembic

1.1 alembic init alembic



```
Windows PowerShell - alembic init

PS D:\python avanzado\ev10\device_systems> alembic init alembic

Creating directory D:\python avanzado\ev10\device_systems\alembic ... done
Creating directory D:\python avanzado\ev10\device_systems\alembic\versions ... done
Generating D:\python avanzado\ev10\device_systems\alembic.ini ... done
Generating D:\python avanzado\ev10\device_systems\alembic\env.py ... done
Generating D:\python avanzado\ev10\device_systems\alembic\README ... done
Generating D:\python avanzado\ev10\device_systems\alembic\script.py.mako ... done
Please edit configuration/connection/logging settings in D:\python avanzado\ev10\device_systems\alembic.ini before proceeding.
```

1.2 alembic revision --autogenerate -m "create devices and loans tables"

Alembic compara los modelos con la base de datos y detecta las tablas `devices` y `loans` junto con sus índices.

```

Windows PowerShell - alembic revision --autogenerate

PS D:\python avanzado\ev10\device_systems> alembic revision --autogenerate -m "create devices and loans tables"

Generating D:\python avanzado\ev10\device_systems\alembic\versions\af3946961193_create_devices_and_loans_tables.py ... done
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.schemas
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.tables
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.types
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.constraints
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.defaults
INFO [alembic.runtime.plugins] setting up autogenerate plugin alembic.autogenerate.comments
INFO [alembic.autogenerate.compare.tables] Detected added table 'devices'
INFO [alembic.autogenerate.compare.constraints] Detected added index 'ix_devices_device_type' on '('device_type',)'
INFO [alembic.autogenerate.compare.constraints] Detected added index 'ix_devices_id' on '('id',)'
INFO [alembic.autogenerate.compare.constraints] Detected added index 'ix_devices_serial_number' on '('serial_number',)'
INFO [alembic.autogenerate.compare.tables] Detected added table 'loans'
INFO [alembic.autogenerate.compare.constraints] Detected added index 'ix_loans_device_id' on '('device_id',)'
INFO [alembic.autogenerate.compare.constraints] Detected added index 'ix_loans_id' on '('id',)'
INFO [alembic.autogenerate.compare.constraints] Detected added index 'ix_loans_status' on '('status',)'
INFO [alembic.autogenerate.compare.constraints] Detected added index 'ix_loans_user_id' on '('user_id',)'

```

1.3 alembic upgrade head

```

Windows PowerShell - alembic upgrade head

PS D:\python avanzado\ev10\device_systems> alembic upgrade head

INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Running upgrade -> 75b8b80c779a, create users table
INFO [alembic.runtime.migration] Running upgrade 75b8b80c779a -> af3946961193, create devices and loans tables

```

1.4 alembic history y alembic current

```
Windows PowerShell - alembic history / current

PS D:\python avanzado\ev10\device_systems> alembic history

75b8b80c779a -> e61a53a3b60c (head), create devices and loans tables
<base> -> 75b8b80c779a, create users table

PS D:\python avanzado\ev10\device_systems> alembic current

e61a53a3b60c (head)
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
```

Revision	Descripcion	Tablas
75b8b80c779a	create users table	users
e61a53a3b60c	create devices and loans tables	devices , loans

2. Estructura de tablas generadas

Las tres tablas con sus constraints y las claves foraneas de `loans` hacia `users` y `devices` .

```
Windows PowerShell - tablas generadas

PS D:\python avanzado\ev10\device_systems> sqlite3 device_systems.db

sqlite> .tables
alembic_version  devices  loans  users

sqlite> .schema users
CREATE TABLE users (
  id INTEGER NOT NULL,
  name VARCHAR(60) NOT NULL,
  email VARCHAR(120) NOT NULL,
  role VARCHAR(20) NOT NULL,
  is_active BOOLEAN NOT NULL,
  created_at DATETIME NOT NULL,
  PRIMARY KEY (id)
);

sqlite> .schema devices
CREATE TABLE devices (
  id INTEGER NOT NULL,
  name VARCHAR(80) NOT NULL,
  serial_number VARCHAR(50) NOT NULL,
  device_type VARCHAR(30) NOT NULL,
  brand VARCHAR(40),
  is_available BOOLEAN NOT NULL,
  created_at DATETIME NOT NULL,
  PRIMARY KEY (id)
);

sqlite> .schema loans
CREATE TABLE loans (
  id INTEGER NOT NULL,
  user_id INTEGER NOT NULL,
  device_id INTEGER NOT NULL,
  loan_date DATETIME NOT NULL,
  return_date DATETIME,
  status VARCHAR(20) NOT NULL,
  PRIMARY KEY (id),
  FOREIGN KEY(device_id) REFERENCES devices (id) ON DELETE CASCADE,
  FOREIGN KEY(user_id) REFERENCES users (id) ON DELETE CASCADE
);
```

3. Capturas de Swagger UI

Documentacion organizada por tags: **Users**, **Devices**, **Loans** y **Root**.

device_systems API 4.0.0 OAuth 2.0

API REST para la gestión de usuarios, dispositivos y préstamos del sistema `device_systems`.

Novedades de esta version

- Migraciones de base de datos con `sequelize`.
- Actualización de `sequelize` a la versión `4.0.0`.
- Actualización de `sequelize` a la versión `4.0.0`.
- Reglas de negocio del préstamo y la devolución de dispositivos.

Contacta Daniel Roman - Agenciat SENA

Users Gestión de usuarios y consulta de sus préstamos.

GET	/users	Lista usuarios
POST	/users	Crear un usuario
GET	/users/{user_id}	Consultar usuario por ID
PUT	/users/{user_id}	Actualizar un usuario por completo
PATCH	/users/{user_id}	Actualizar parcialmente un usuario
DELETE	/users/{user_id}	Eliminar un usuario
GET	/users/{user_id}/loans	Préstamos de un usuario

Devices Gestión de dispositivos tecnológicos disponibles para préstamo, con filtros por tipo, marca, disponibilidad y búsqueda libre.

GET	/devices	Lista dispositivos
POST	/devices	Registrar un dispositivo
GET	/devices/{device_id}	Consultar dispositivo por ID
PUT	/devices/{device_id}	Actualizar un dispositivo por completo
PATCH	/devices/{device_id}	Actualizar parcialmente un dispositivo
DELETE	/devices/{device_id}	Eliminar un dispositivo
GET	/devices/{device_id}/loans	Historial de préstamos del dispositivo

Loans Gestión de préstamos: registro, devolución y consultas con joins entre usuarios y dispositivos.

GET	/loans	Lista préstamos
POST	/loans	Registrar un préstamo
GET	/loans/details	Lista préstamos con información relacionada
GET	/loans/{loan_id}	Consultar un préstamo por ID
PATCH	/loans/{loan_id}	Actualizar el estado de un préstamo
PATCH	/loans/{loan_id}/return	Registrar la devolución de un préstamo

Root Información general y estado de la API.

GET	/	Información de la API
GET	/health	Estado del servicio

Schemas

DeviceCreate	Expend al: object
DeviceListResponse	Expend al: object
DeviceMini	Expend al: object
DevicePatch	Expend al: object
DeviceResponse	Expend al: object
DeviceType	Expend al: string
DeviceUpdate	Expend al: object
HTTPValidationError	Expend al: object
LoanCreate	Expend al: object
LoanDetailListResponse	Expend al: object
LoanDetailResponse	Expend al: object
LoanListResponse	Expend al: object
LoanResponse	Expend al: object
LoanStatus	Expend al: string
LoanUpdate	Expend al: object
MessageResponse	Expend al: object
OrderUsuarios	Expend al: string
UserCreate	Expend al: object
UserListResponse	Expend al: object
UserMini	Expend al: object
UserPatch	Expend al: object
UserResponse	Expend al: object
UserRole	Expend al: string
UserUpdate	Expend al: object
ValidationError	Expend al: object

4. Evidencia de creacion de usuario, dispositivo y prestamo

4.1 Crear usuario — 201

POST /users Crear un usuario

Registra un usuario en la base de datos validando los datos con Pydantic y comprobando que el correo no exista (constraint unique).

Parameters

No parameters

Request body required

application/json

Edit Value Schema

```
{
  "name": "Diana Lopez",
  "email": "diana@sena.edu.co",
  "role": "support",
  "is_active": true
}
```

Execute

Clear

Responses

Curl

curl -X 'POST' \
 'http://127.0.0.1:8010/users' \
 -H 'accept: application/json' \
 -H 'Content-Type: application/json' \
 -d '{
 "name": "Diana Lopez",
 "email": "diana@sena.edu.co",
 "role": "support",
 "is_active": true
 }'

Request URL

http://127.0.0.1:8010/users

Server response

Code	Details
201	Response body <pre>{ "id": 4, "name": "Diana Lopez", "email": "diana@sena.edu.co", "role": "support", "is_active": true, "created_at": "2026-09-12T17:51:33.623754" }</pre> Download Response headers <pre>content-length: 133 content-type: application/json date: Sat, 12 Sep 2026 17:51:33 GMT location: /users/4 server: uvicorn x-api-version: 4.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
201	Usuario creado Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 0, "name": "string", "email": "user@example.com", "role": "admin", "is_active": true, "created_at": "2026-09-12T17:51:33.7632" }</pre>	

 No links || 400 | Solicitud invalida | No links |
| 422 | Datos invalidos | No links |

4.2 Crear dispositivo — 201

POST /devices Registrar un dispositivo

Crea un dispositivo validando que el numero de serie sea unico.

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "name": "Monitor LG 24",
  "serial_number": "lg-2024-030",
  "device_type": "monitor",
  "brand": "LG"
}
```

Execute

Clear

Responses

Curl

curl -X 'POST' \
 'http://127.0.0.1:8010/devices' \
 -H 'accept: application/json' \
 -H 'content-type: application/json' \
 -d '{
 "name": "Monitor LG 24",
 "serial_number": "lg-2024-030",
 "device_type": "monitor",
 "brand": "LG"
 }'

Request URL

http://127.0.0.1:8010/devices

Server response

Code

Details

201

Response body

```
{
  "id": 5,
  "name": "Monitor LG 24",
  "serial_number": "LG-2024-030",
  "device_type": "monitor",
  "brand": "LG",
  "is_available": true,
  "created_at": "2026-09-12T17:51:37.333431"
}
```

Response headers

content-length: 160
content-type: application/json
date: Sat, 12 Sep 2026 17:51:36 GMT
location: /devices/5
server: uvicorn
x-api-version: 4.0
x-app-name: device_systems

Responses

Code	Description	Links
201	Dispositivo creado Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 0, "name": "string", "serial_number": "string", "device_type": "laptop", "brand": "string", "is_available": true, "created_at": "2026-09-12T17:51:37.4322" }</pre>	No links
400	Solicitud invalida	No links
422	Datos invalidos	No links

4.3 Crear prestamo — 201

Al crear el prestamo se devuelve la informacion del usuario y del dispositivo, y el dispositivo pasa a `is_available = false`.

Pagina 7 de 18

POST

/loans Registrar un prestamo

Crea un prestamo validando que el usuario exista, que el dispositivo exista y que este disponible. Al crearlo, el dispositivo pasa a `is_available = false`.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value

Schema

{
 "user_id": 1,
 "device_id": 1
}

Execute

Clear

Responses

Curl

curl -X 'POST' \
 'http://127.0.0.1:8018/loans' \
 -H 'accept: application/json' \
 -H 'content-type: application/json' \
 -d '{
 "user_id": 1,
 "device_id": 1
 }'

Request URL

http://127.0.0.1:8018/loans

Server response

Code

Details

201

Response body

{
 "loan_id": 1,
 "status": "active",
 "loan_date": "2026-09-12T17:51:40.788879",
 "return_date": null,
 "user": {
 "id": 1,
 "name": "Ana Perez",
 "email": "anaperez@edu.co"
 },
 "device": {
 "id": 1,
 "name": "Laptop Lenovo ThinkPad",
 "serial_number": "LEN-2024-001",
 "device_type": "laptop"
 }
}

Response headers

content-length: 356
content-type: application/json
date: Sat, 12 Sep 2026 17:51:39 GMT
location: /loans/1
server: uvicorn
x-application: d.0
x-app-name: device_systems

Responses

Code

Description

Links

201

Prestamo creado

No links

Media type

application/json

Controls

Accept header

Example Value

Schema

{
 "loan_id": 0,
 "status": "active",
 "loan_date": "2026-09-12T17:51:40.8672",
 "return_date": "2026-09-12T17:51:40.8672",
 "user": {
 "id": 1,
 "name": "string",
 "email": "user@example.com"
 },
 "device": {
 "id": 1,
 "name": "string",
 "serial_number": "string",
 "device_type": "laptop"
 }
}

404

Recurso no encontrado

No links

409

Regla de negocio incumplida

No links

422

Validation Error

No links

Media type

application/json

Example Value

Schema

{
 "detail": [
 {
 "loc": [
 "string",
 0
],
 "msg": "string",
 "type": "string",
 "input": "string",
 "ctx": {}
 }
]
}

4.4 Intento de prestar un dispositivo no disponible — 409 Conflict

POST

/loans

Registrar un prestamo

Crea un prestamo validando que el usuario exista, que el dispositivo exista y que este disponible. Al crearlo, el dispositivo pasa a `is_available = false`.

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

Edit Value

Schema

```
{
  "user_id": 2,
  "device_id": 1
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8018/loans' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": 2,
    "device_id": 1
  }'
```

Request URL

http://127.0.0.1:8018/loans

Server response

Code

Details

409

Error: Conflict

Response body

```
{
  "error": true,
  "message": "El dispositivo no esta disponible",
  "status_code": 409,
  "path": "/loans"
}
```

Download

Response headers

```
content-length: 94
content-type: application/json
date: Sat, 12 Sep 2020 17:52:43 GMT
server: uvicorn
x-api-version: 4.0
x-app-name: device_systems
```

Responses

Code

Description

Links

201

Prestamo creado

No links

Media type

application/json

Controls Accept header.

Example Value

Schema

```
{
  "loan_id": 0,
  "status": "active",
  "loan_date": "2020-09-12T17:51:44.516Z",
  "return_date": "2020-09-12T17:51:44.516Z",
  "user": {
    "id": 1,
    "name": "string",
    "email": "user@example.com"
  },
  "device": {
    "id": 0,
    "name": "string",
    "serial_number": "string",
    "device_type": "laptop"
  }
}
```

404

Recurso no encontrado

No links

409

Regla de negocio incumplida

No links

422

Validation Error

No links

Media type

application/json

Example Value

Schema

```
{
  "detail": {
    "loc": [
      "string",
      0
    ],
    "msg": "string",
    "type": "string",
    "input": "string",
    "ctx": {}
  }
}
```

5. Evidencia de consultas con joins

5.1 GET /loans/details — prestamos con usuario y dispositivo

GET /loans/details

Listar prestamos con informacion relacionada

Consulta con joins: devuelve cada prestamo junto con los datos basicos del usuario y del dispositivo. Acepta los mismos filtros que [GET /loans](#).

Parameters

Cancel

Name	Description
status	--
string (string null) (query)	
user_id	user_id
integer (integer null) (query)	required: 1
device_id	device_id
integer (integer null) (query)	required: 1
user_email	user_email
string (string null) (query)	
device_type	--
string (string null) (query)	
search	search
string (string null) (query)	

ExecuteClear

Responses

curl -X GET '\nhttp://127.0.0.1:8080/loans/details' \n-H 'accept: application/json'

Request URL

http://127.0.0.1:8080/loans/details

Server response

Code

Details

200

Response body

```
{
  "total": 2,
  "data": [
    {
      "loan_id": 1,
      "status": "Active",
      "loan_date": "2020-09-12T17:11:45.955295",
      "return_date": null,
      "user": {
        "id": 1,
        "name": "Carla Gomez",
        "email": "carla@demio.886.co"
      },
      "device": {
        "id": 1,
        "name": "Trayector Epsom PowerLite",
        "serial_number": "EPS-2023-007",
        "device_type": "projector"
      }
    },
    {
      "loan_id": 1,
      "status": "Active",
      "loan_date": "2020-09-12T17:11:45.788879",
      "return_date": null,
      "user": {
        "id": 1,
        "name": "Ana Perez",

```

Download

Response headers

```
content-length: 546
content-type: application/json
date: Sat, 12 Sep 2020 17:11:47 GMT
server: nodejs
x-api-version: 4.0
x-app-name: device_systems
```

Responses

Code

Description

Links

200

Prestamos con usuario y dispositivo

No links

Media type

application/json

Controls Accept header

Example Value

Schema

```
{
  "total": 2,
  "data": [
    {
      "loan_id": 1,
      "status": "Active",
      "loan_date": "2020-09-12T17:11:47.88827",
      "return_date": "2020-09-12T17:11:47.88827",
      "user": {
        "name": "string",
        "email": "user@sample.com"
      },
      "device": {
        "id": 1,
        "name": "string",
        "serial_number": "string",
        "device_type": "laptop"
      }
    }
  ]
}
```

422

Validation Error

No links

Media type

application/json

Example Value

Schema

```
{
  "detail": {
    "loc": [
      "string",
      "string"
    ],
    "msg": "string",
    "type": "string",
    "input": "string",
    "ctx": {}
  }
}
```

5.2 GET /users/{user_id}/loans — prestamos de un usuario

GET

/users/{user_id}/loans

Prestamos de un usuario

Consulta con join: devuelve los prestamos del usuario junto con la informacion del dispositivo asociado.

Parameters

Cancel

Name	Description
user_id required	Identificador del usuario
integer (path)	
	minimum: 1

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8010/users/1/loans' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8010/users/1/loans

Server response

Code	Details
200	Response body <pre>{ "total": 1, "data": [{ "loan_id": 1, "status": "active", "loan_date": "2026-09-12T17:51:48.788579", "return_date": null, "user": { "id": 1, "name": "Ana Perez", "email": "ana@sena.edu.co" }, "device": { "id": 1, "name": "Laptop Lenovo ThinkPad", "serial_number": "LEN-2024-001", "device_type": "laptop" } }] }</pre> Response headers <pre>content-length: 277 content-type: application/json date: Sat, 12 Sep 2026 17:51:58 GMT server: uvicorn x-api-version: 4.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
200	Prestamos del usuario Media type application/json Controls Accept header. Example Value Schema <pre>{ "total": 0, "data": [{ "loan_id": 0, "status": "active", "loan_date": "2026-09-12T17:51:58.7162", "return_date": "2026-09-12T17:51:58.7162", "user": { "id": 0, "name": "string", "email": "user@example.com" }, "device": { "id": 0, "name": "string", "serial_number": "string", "device_type": "laptop" } }] }</pre>	No links
404	Usuario no encontrado	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

5.3 GET /devices/{device_id}/loans — historial del dispositivo

GET

/devices/{device_id}/loans

Historial de prestamos del dispositivo

Consulta con join: prestamos del dispositivo con usuario asociado.

Parameters

Cancel

Name	Description
device_id required	Identificador del dispositivo
integer (path)	
	1
	minimum: 1

Execute

Clear

Responses

Curl

curl -X 'GET' \ 'http://127.0.0.1:8010/devices/1/loans' \ -H 'accept: application/json'

Request URL

http://127.0.0.1:8010/devices/1/loans

Server response

Code	Details
200	Response body <pre>{ "total": 1, "data": [{ "loan_id": 1, "status": "returned", "loan_date": "2026-09-12T17:52:08.988579", "return_date": "2026-09-12T17:52:02.156576", "user": { "id": 1, "name": "Ana Perez", "email": "ana@sena.edu.co" }, "device": { "id": 1, "name": "Laptop Lenovo ThinkPad", "serial_number": "LEN-2024-001", "device_type": "laptop" } }] }</pre> Download Response headers content-length: 303 content-type: application/json date: Sat 12 Sep 2026 17:52:08 GMT server: uvicorn x-api-version: 4.0 x-app-name: device_systems

Responses

Code	Description	Links
200	Historial de prestamos Media type application/json Controls Accept header. Example Value Schema <pre>{ "total": 0, "data": [{ "loan_id": 0, "status": "active", "loan_date": "2026-09-12T17:52:09.0912", "return_date": "2026-09-12T17:52:09.0912", "user": { "id": 0, "name": "string", "email": "user@example.com" }, "device": { "id": 0, "name": "string", "serial_number": "string", "device_type": "laptop" } }] }</pre>	

No links

 No links || 404 | Dispositivo no encontrado | No links |
| 422 | Validation Error Media type application/json Example Value | Schema ``` { "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] } ``` |

No links

6. Evidencia de filtros aplicados

6.1 Filtrar por estado — GET /loans?status=active

GET /loans Listar prestamos

Listar los prestamos registrados. Filtros disponibles: status, user_id, device_id, user_email (join con users), device_type y search (join con devices).

Parameters

Cancel

Name	Description
status	Estado del prestamo
string (string null) (query)	active
user_id	ID del usuario
integer (integer null) (query)	user_id
device_id	ID del dispositivo
integer (integer null) (query)	device_id
user_email	Correo del usuario
string (string null) (query)	user_email
device_type	Tipo de dispositivo
string (string null) (query)	-
search	Nombre del dispositivo
string (string null) (query)	search

ExecuteClear

Responses

Curl

curl -X 'GET' \n'http://127.0.0.1:8010/loans?status=active' \n-H 'accept: application/json'

Request URL

http://127.0.0.1:8010/loans?status=active

Server response

Code

Details

200

Response body

```
{
  "total": 2,
  "data": [
    {
      "id": 1,
      "user_id": 1,
      "device_id": 1,
      "loan_date": "2026-09-12T17:51:49.956291",
      "return_date": null,
      "status": "active"
    },
    {
      "id": 2,
      "user_id": 1,
      "device_id": 1,
      "loan_date": "2026-09-12T17:51:49.798879",
      "return_date": null,
      "status": "active"
    }
  ]
}
```

Response headers

content-length: 246
content-type: application/json
date: Sat, 12 Sep 2026 17:51:59 GMT
server: unicorn
x-api-version: 4.0
x-app-name: device_systems
x-total-loans: 2

Responses

Code	Description	Links
200	Listado de prestamos	No links
422	Validation Error	No links

Pagina 13 de 18

6.2 Filtrar por tipo de dispositivo — GET /loans/details?device_type=laptop

GET /loans/details

Listar prestamos con informacion relacionada

Consulta con joins: devuelve cada prestamo junto con los datos basicos del usuario y del dispositivo. Acepta los mismos filtros que GET /loans.

Parameters

Cancel

Name	Description
status	-- string (string null) (query)
user_id	user_id integer (integer null) (query) mininum: 1
device_id	device_id integer (integer null) (query) mininum: 1
user_email	user_email string (string null) (query)
device_type	laptop string (string null) (query)
search	search string (string null) (query)

Execute

Clear

Responses

Curl

curl -X 'GET' \nhttp://127.0.0.1:8080/loans/details?device_type=laptop' \n-H 'accept: application/json'

Request URL

http://127.0.0.1:8080/loans/details?device_type=laptop

Server response

Code

Details

200

Response body

{\n \"total\": 1,\n \"data\": [\n {\n \"loan_id\": 1,\n \"status\": \"active\",\n \"loan_date\": \"2026-09-12T17:51:49.788379\",\n \"return_date\": null,\n \"user\": {\n \"id\": 1,\n \"name\": \"Ana Perez\",\n \"email\": \"ana@ena.edu.co\"\n },\n \"device\": {\n \"id\": 1,\n \"name\": \"Laptop Lenovo ThinkPad\",\n \"serial_number\": \"LEN-2024-001\",\n \"device_type\": \"laptop\"\n }\n }\n]\n}

Response headers

content-length: 277\ncontent-type: application/json\ndate: Sat, 12 Sep 2026 17:51:54 GMT\nserver: uvicorn\nx-api-version: 4.0\nx-app-name: device_systems

Responses

Code

Description

Links

200

Prestamos con usuario y dispositivo

No links

Media type

application/json

Controls Accept header

Example Value

Schema

{\n \"total\": 1,\n \"data\": [\n {\n \"loan_id\": 1,\n \"status\": \"active\",\n \"loan_date\": \"2026-09-12T17:51:55.2822\",\n \"return_date\": \"2026-09-12T17:51:55.2822\",\n \"user\": {\n \"id\": 1,\n \"name\": \"string\",\n \"email\": \"user@example.com\"\n },\n \"device\": {\n \"id\": 1,\n \"name\": \"string\",\n \"serial_number\": \"string\",\n \"device_type\": \"laptop\"\n }\n }\n]\n}

422

Validation Error

No links

Media type

application/json

Example Value

Schema

{\n \"detail\": {\n \"loc\": [\n \"string\",\n 0\n],\n \"msg\": \"string\",\n \"type\": \"string\",\n \"input\": \"string\",\n \"ctx\": {\n }\n }\n}

Los filtros se construyen dinamicamente con `join()`, `where()`, `ilike()` y `and_()`:

Pagina 14 de 18

```
if user_email is not None:
    consulta = consulta.join(User, Loan.user_id == User.id)
    condiciones.append(User.email.ilike(f"%{user_email}%"))

if condiciones:
    consulta = consulta.where(and_(*condiciones))
```

7. Evidencia de devolucion de dispositivo

7.1 PATCH /loans/{loan_id}/return — 200 OK

El prestamo queda en `returned` y se asigna `return_date`.

PATCH

/loans/{loan_id}/return

Registrar la devolucion de un prestamo

Marca el prestamo como **returned** , asigna la fecha de devolucion y vuelve a dejar el dispositivo disponible. Si el prestamo ya fue devuelto responde 409 Conflict.

Parameters

Cancel

Name	Description
loan_id * required	Identificador del prestamo
integer	
(path)	
	1
	minimum: 1

Execute

Clear

Responses

Curl

```
curl -X 'PATCH' \
  http://127.0.0.1:8010/loans/1/return \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8010/loans/1/return

Server response

Code	Details
200	Response body <pre>{ "loan_id": 1, "status": "returned", "loan_date": "2026-09-12T17:51:40.708879", "return_date": "2026-09-12T17:52:02.156570", "user": { "id": 1, "name": "Ana Perez", "email": "ana@sena.edu.co" }, "device": { "id": 1, "name": "Laptop lenovo ThinkPad", "serial_number": "LEN-2024-001", "device_type": "laptop" } }</pre> Download

Response headers

```
content-length: 292
content-type: application/json
date: Sat, 12 Sep 2026 17:52:01 GMT
server: uvicorn
x-api-version: 4.0
x-app-name: device_systems
```

Responses

Code	Description	Links
200	Prestamo devuelto Media type application/json Controls Accept header. Example Value Schema <pre>{ "loan_id": 0, "status": "active", "loan_date": "2026-09-12T17:52:02.2862", "return_date": "2026-09-12T17:52:02.2862", "user": { "id": 0, "name": "string", "email": "user@example.com" }, "device": { "id": 0, "name": "string", "serial_number": "string", "device_type": "laptop" } }</pre>	

No links

 No links || 404 | Recurso no encontrado | No links |
| 409 | Regla de negocio incumplida | No links |
| 422 | Validation Error | No links |

Media type

application/json

Example Value

Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

7.2 El dispositivo vuelve a estar disponible

GET

/devices/{device_id}

Consultar dispositivo por ID

Parameters

Cancel

Name	Description
device_id ★ required	Identificador del dispositivo
integer	
(path)	
	1
	minimum: 1

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8010/devices/1' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8010/devices/1
```

Server response

Code	Details
200	Response body <pre>{ "id": 1, "name": "Laptop Lenovo ThinkPad", "serial_number": "LEN-2024-001", "device_type": "laptop", "brand": "Lenovo", "is_available": true, "created_at": "2026-09-12T17:34:58.903927" }</pre> Download Response headers <pre>content-length: 173 content-type: application/json date: Sat, 12 Sep 2026 17:52:05 GMT server: uvicorn x-api-version: 4.0 x-app-name: device_systems</pre>

Responses

Code	Description	Links
200	Dispositivo encontrado	No links
	Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": 0, "name": "string", "serial_number": "string", "device_type": "laptop", "brand": "string", "is_available": true, "created_at": "2026-09-12T17:52:05.9462" }</pre>	
404	Dispositivo no encontrado	No links
422	Validation Error	No links
	Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	

8. Relaciones entre modelos



```
# user_model.py / device_model.py
loans: Mapped[List["Loan"]] = relationship("Loan", back_populates="user")
loans: Mapped[List["Loan"]] = relationship("Loan", back_populates="device")

# loan_model.py
user_id: ForeignKey("users.id")
device_id: ForeignKey("devices.id")
user: Mapped[User] = relationship("User", back_populates="loans")
device: Mapped[Device] = relationship("Device", back_populates="loans")
```

9. Resumen de pruebas

#	Prueba	Esperado	Resultado
1	Ejecutar migraciones con Alembic	2 revisiones aplicadas	OK
2	Crear usuario	201	OK
3	Crear dispositivo	201	OK
4	Crear prestamo	201	OK
5	Prestar un dispositivo no disponible	409	OK
6	Listar prestamos con informacion relacionada	200	OK
7	Filtrar prestamos por estado	200	OK
8	Filtrar prestamos por tipo de dispositivo	200	OK
9	Consultar prestamos de un usuario	200	OK
10	Devolver un dispositivo	200	OK
11	Validar que el dispositivo vuelva a estar disponible	is_available: true	OK
12	Consultar historial de prestamos del dispositivo	200	OK

10. Reflexion sobre migraciones, relaciones y consultas avanzadas

Antes de Alembic, cada cambio en los modelos significaba borrar la base de datos y volver a crearla, lo que en un proyecto real equivale a perder la informacion. Con Alembic la estructura tiene historia: cada cambio queda en un archivo de `versions/`, se aplica con `upgrade head` y se revierte con `downgrade`.

Las relaciones cambiaron la forma de pensar los datos. `loans` conecta `users` con `devices` mediante `ForeignKey`, y las `relationship()` con `back_populates` permiten navegar en ambos sentidos. La integridad referencial garantiza que nunca exista un prestamo apuntando a un usuario o dispositivo inexistente.

Las consultas con joins son las que dan valor real a la API: `GET /loans/details` responde en una sola llamada quien tiene prestado que equipo, y `joinedload()` evita el problema de las N+1 consultas. Junto con los filtros dinamicos (`ilike`, `and_`, `or_`) y las reglas de negocio con `409 Conflict`, la API dejo de ser un CRUD y paso a modelar un proceso real de prestamos.